Solver Manual

SchrodingerSolver

Elmer FEM 9.0 — SchrodingerSolve

Module: `SchrodingerSolve` · **Subroutine:** `SchrodingerSolver`

Source file: `01_simulacoes/schrodinger_P1_2/fonte/SchrodingerSolver.F90` (460 lines)

Language: Fortran 90 · **Elmer version:** 9.0

> *This manual describes the **implementation and usage** of the solver. For the physics and the keywords, see `Manual_Model_Schrodinger.md`.*

Contents

1. Overview
2. Code structure
3. Interface with Elmer
4. Compilation and installation
5. Usage — minimal SIF
6. Output and log interpretation
7. Matrix export
8. Implementation peculiarities
9. Error diagnostics
10. Extending the solver

---

1. Overview

`SchrodingerSolver` is an Elmer **user solver** (it is not part of the official distribution). It solves the generalised eigenproblem `A ψ = E M ψ` in 1D, 2D and 3D.

How it differs from the native solvers

Aspect	Elmer native solvers	`SchrodingerSolver`
Assembly	**sparse global** matrix via `DefaultUpdateEquations`	**dense local** matrices (`ALLOC`
Eigensolver	`EigenSolve.F90` (ARPACK/iterative)	**`DSYGV`** (LAPACK, dense)
Parallelism	native MPI	**none** (see §8.4)
Dependencies	Elmer infrastructure	only `DefUtils` + LAPACK

Rationale: Elmer's internal ARPACK eigensolver showed problems on this version/platform, and the independent `DSYGV` route proved reliable and verifiable. The price is the lack of parallelism and sparse assembly — compensated by the matrix export (§7).

Verified scope

		Dimension	Status	Verification
1D	✅			error 3.3e−04; agreement 4.2e−11 with an independent Python sol
2D	✅			$O(h^2)$ convergence, order 1.99
3D	✅			correct triple degeneracy
$V \neq 0$ 1D/2D/3D	✅			harmonic, Morse, double well, sinusoidal

---

2. Code structure

```
SchrodingerSolver.F90
├── SUBROUTINE SchrodingerSolver( Model, Solver, dt, Transient )   [line 50]
│   ├── 1) parameter reading        (Eigen System Values, Apply Dirichlet, ...)
│   ├── 2) boundary node detection  (geometric, independent of BC)
│   ├── 3) potential V reading      (Potential / Potential X,Y,Z / V=0)
│   ├── 4) assembly: loop over elements
│   │       ├── Gauss quadrature (Integ)
│   │       ├── Stiff    = ∫ ∇Nᵢ·∇Nⱼ
│   │       ├── Mass     = ∫ NᵢNⱼ
│   │       └── PotMass  = ∫ Nᵢ V Nⱼ
│   ├── 5) A = K + Mᵃ ;  B = M
│   ├── 6) elimination of fixed DOFs -> Af, Bf
│   ├── 7) DSYGV(Af, Bf) -> eigenvalues and eigenvectors
│   ├── 8) write eigenvalues; mode 1 into the output field
│   ├── 9) optional export of A, M, fixed
│   └── 10) deallocation
└── SUBROUTINE ExportaMatriz( nome, Mat, n, arquivo )              [line 432]
```

Main variables

Variable	Type	Role
`A, B`	`REAL, ALLOCATABLE (:,:)`	dense global matrices (operator and mass)
`Af, Bf`	`REAL, ALLOCATABLE (:,:)`	reduced version (free DOFs only)
`Stiff, Mass, PotMass`	`REAL, ALLOCATABLE (:,:)`	element matrices
`EigenVals`	`REAL, ALLOCATABLE (:)`	eigenvalues (the energies)
`xfree`	`REAL, ALLOCATABLE (:,:)`	eigenvectors, column `i` = mode `i`
`Fixed`	`LOGICAL, ALLOCATABLE (:)`	`.TRUE.` for nodes with $\psi = 0$
`IdxFree`	`INTEGER, ALLOCATABLE (:)`	(global) indices of the free DOFs

Variable	Type	Role
`Vnodal`	`REAL, ALLOCATABLE (:)`	potential evaluated at every node

---

3. Interface with Elmer

Required signature

```
SUBROUTINE SchrodingerSolver( Model, Solver, dt, Transient )
  USE DefUtils
  TYPE(Solver_t) :: Solver
  TYPE(Model_t)  :: Model
  REAL(KIND=dp)  :: dt
  LOGICAL        :: Transient
```

The signature follows the Elmer standard for user solvers. `USE DefUtils` provides `ListGetInteger`, `ListGetLogical`, `ListGetString`, `ListGetReal`, `ListCheckPresent`, `CoordinateSystemDimension`, `GetElementNodes`, `GaussPoints`, `Info`, `Message`, `I2S`, `MAX_NAME_LEN`.

> ***`USE DefUtils` exports a subroutine named `Info`.*** *If the code declares a local variable `INFO` (as the LAPACK return argument), the names collide and compilation fails with cascading errors.* ***Use another name*** *— in this solver the LAPACK variable is called* ***`info_lap`****.*

Data flow

```
SIF  ⟶  Model % BodyForces(1) % Values  ⟶  ListGetReal('Potential')
     ⟶  Solver % Values                 ⟶  ListGetInteger/Logical/String
     ⟶  Model % NumberOfBulkElements, Element % NodeIndexes, Mesh % Nodes
     ⟵  Solver % Variable % Values      ⟵  eigenfunction (mode 1)
```

Problem dimension

```
dim = CoordinateSystemDimension()
```

`dim` comes from `Coordinate System` in the SIF, **not** from the mesh. It is therefore mandatory that the two agree:

`Coordinate System`	Mesh	Result
`Cartesian 2D`	2D	✓

Coordinate System	Mesh	Result
Cartesian 2D	3D	ERROR:: ElementMetric: Degenerate 3D element

---

4. Compilation and installation

4.1 Prerequisite: a compiled Elmer 9.0

The solver needs the Elmer **`.mod` modules** and **`libelmersolver.so`**.

In this project there is a GNU build with MUMPS:

```
~/elmer_build/build_mumps/          # build tree
~/elmer_build/install_mumps/        # installation
```

The full build recipe is in `01_simulacoes/build_elmer/LEIA-ME.md`.

4.2 Compiling the solver

On the headnode (compilation is allowed; simulation is not):

```
source /etc/profile.d/modules.sh
module load gnu12 openmpi4 openblas
export LC_ALL=C

BLD=$HOME/elmer_build/build_mumps
INST=$HOME/elmer_build/install_mumps

# optional: packaged C headers (compute nodes have no /usr/include)
export CPATH="$HOME/sysinclude/usr/include${CPATH:+:$CPATH}"

cd ~/schrodinger_P1_2/build_mumps
mpif90 -c SchrodingerSolver.F90 -o SchrodingerSolver.o \
   -fallow-argument-mismatch -DCONTIG= -DHAVE_EXECUTECOMMANDLINE \
   -DUSE_ISO_C_BINDINGS -O2 -fPIC -I$BLD/fmodules

mpif90 -shared SchrodingerSolver.o -o SchrodingerSolver.so \
   -L$BLD/fem/src -lelmersolver

cp -f SchrodingerSolver.so $INST/share/elmersolver/lib/
```

4.3 Flags explained

Flag	Why
`-fallow-argument-mismatch`	gfortran 12 is strict; the Elmer API has implicit interfaces
`-DCONTIG=`	Elmer uses this macro for `CONTIGUOUS`; empty = disabled

Flag	Why
`-DHAVE_EXECUTECOMMANDLINE`	enables command-line execution in the library
`-DUSE_ISO_C_BINDINGS`	enables the Elmer C bindings
`-fPIC`	**mandatory** to produce a `.so`
`-I$BLD/fmodules`	where the Elmer `.mod` files are

4.4 Runtime environment

```
export ELMER_HOME=$HOME/elmer_build/install_mumps
export ELMER_SOLVER_HOME=$ELMER_HOME/share/elmersolver
export LD_LIBRARY_PATH=$HOME/elmer_build/install_mumps/lib/elmersolver:
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/opt/ohpc/pub/libs/gnu12/openmpi4/mumps/5.2.1/lib
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/opt/ohpc/pub/libs/gnu12/openmpi4/scalapack/2.2.0/lib
export ES=$ELMER_HOME/bin/ElmerSolver
```

4.5 Checklist

```
# 1) is the .so installed?
ls -la $ELMER_SOLVER_HOME/lib/SchrodingerSolver.so

# 2) does Elmer load the module?
$ES caso.sif 2>&1 | grep -i "SchrodingerSolver"
# expected: Loading user function library: [SchrodingerSolver]...[SchrodingerSolver_Init0]
```

---

5. Usage — minimal SIF

```
Header
  Mesh DB "." "malha"
End

Simulation
  Coordinate System = Cartesian 1D
  Simulation Type = Steady State
  Steady State Max Iterations = 1
  Max Output Level = 5
End

Body 1
  Equation = 1
  Material = 1
End

Material 1
  Density = 1.0
End

Equation 1
  Active Solvers(1) = 1
```

```
End

Solver 1
  Equation = "Schrodinger"
  Procedure = "SchrodingerSolver" "SchrodingerSolver"
  Variable = "psi"
  Variable DOFs = 1
  Eigen System Values = 5
  Apply Dirichlet = Logical True
  Linear System Solver = Direct
  Linear System Direct Method = "umfpack"
End
```

With a `Potential` (1D), add the `Body Force = 1` reference to `Body 1` and the section:

```
Body Force 1
  Potential = Variable Coordinate 1
    Real MATC "1.0*tx*tx"
End
```

Running

```
$ES caso.sif > run.log 2>&1
grep "E(" run.log
```

---

6. Output and log interpretation

Header

```
SchrodingerSolver: ======================================
SchrodingerSolver: Solver de Schrodinger (FEM) iniciado
SchrodingerSolver: Numero de autovalores pedidos: 5
SchrodingerSolver: Dimensao do problema: 2
SchrodingerSolver: ======================================
```

Boundary and potential

```
SchrodingerSolver: Nos de contorno (Dirichlet psi=0): 160
SchrodingerSolver: V(x) lido como campo nodal de Body Force.
SchrodingerSolver:   V_min =   0.000000E+00   V_max =   5.000000E+01
```

Line	Meaning
`Nos de contorno`	how many nodes received $\psi = 0$ (geometric detectio
`V(x) lido como campo nodal de Body Force`	the potential came from `Potential`
`V(x,y,z) construido de Potential X/Y/Z`	it came from the components

Line	Meaning
`Sem potencial (V = 0)`	**no** potential was found
`V_min` / `V_max`	**quick check** that the potential is correct

> ***Always check `V_min/V_max`.*** *This solver once had a defect in which the potential was read from an invalid section and `V = 0` was used **silently** — see `docs/VALIDACAO_POTENCIAL.md`. The `Sem potencial (V = 0)` message was added precisely to make that visible.*

Result

```
SchrodingerSolver: DOFs livres (apos eliminacao): 1521
SchrodingerSolver: --- AUTOVALORES (energias) ---
SchrodingerSolver:  E(   1) =   1.028585384356E+00
SchrodingerSolver:  E(   2) =   2.043966018888E+00
SchrodingerSolver:  E(   3) =   2.043966018888E+00
SchrodingerSolver: Autoproblema resolvido (DSYGV).
SchrodingerSolver: Concluido.
```

If `DSYGV retornou info = <value>` appears, the diagonalisation failed. `info > 0` indicates QR convergence failure; `info < 0` indicates an invalid argument.

Sensitivity to `Max Output Level`

The solver reports at **`Level=4`**. With `Max Output Level = 3` **all** of the output above is suppressed — the solver appears to do nothing.

---

7. Matrix export

7.1 Activation

```
Solver 1
  ...
  Exportar Matrizes = Logical True
  Exportar Prefixo = String "schrod"
End
```

7.2 Files written

`schrod_A.dat` — the operator $A = K + M^a$:

```
% A  n=441  nnz=3721
441 441 3721
```

```
0 0   5.9583333333333357E+00
0 1  -2.0833333333333330E+00
...
```

Format: `n n nnz` then `i j value`, with `i`, `j` **0-based**.

`schrod_M.dat` — the mass matrix, same format.

`schrod_fixed.dat` — the Dirichlet DOFs:

```
80
0
1
...
```

First line = count; following lines = indices (0-based).

7.3 Solving the exported system

```
python3 01_simulacoes/schrodinger_P1_2/scripts/resolve_exportado.py schrod 5
```

Output verified **identical** to the internal `DSYGV`:

	E(1)	E(2)
Elmer (`DSYGV`)	1.113644656498	2.172208383659
Exported + sparse ARPACK	1.113644656498	2.172208383659

with **52× less** memory. For large meshes this is the recommended route.

7.4 Why export

`DSYGV` is dense: memory `O(n²)`, cost `O(n³)`. Measurements:

DOF	dense	sparse	gain
6 561	41.06 s	0.05 s	821×
14 641	491.60 s	0.12 s	4 097×
39 601	infeasible (37.6 GB)	0.41 s	—

---

8. Implementation peculiarities

8.1 `ListGetReal` is a function returning `F(N)`

Unlike `ListGetConstReal` (scalar), reading a **nodal field** requires:

```
Vtmp = ListGetReal( BFList, 'Potential', nNodes, NodeIdx, Found )
```

- it is a **function** (not a subroutine): `CALL ListGetReal(...)` **does not compile**
- the result is a **fixed-size array** `REAL :: F(N)`, not a pointer — so

`Vtmp => ListGetReal(...)` is not allowed

- `GetRealValues` (a variant) depends on `GetCurrentElement` and **cannot** be

used outside the assembly loop

8.2 Name collisions in Fortran

Fortran is **case-insensitive**. This project hit two real collisions:

Intended name	Collided with	Fix
`BF` (Body Force)	`Bf` (reduced matrix)	renamed to **`BFList`**
`INFO` (LAPACK)	`Info` (subroutine from `DefUtils`)	renamed to **`info_lap`**

8.3 Dirichlet elimination, not penalisation

Fixed DOFs are **eliminated** from the matrix (`Af`, `Bf` containing only the free rows/columns), not replaced by `diagonal = 1`. Penalisation introduces spurious eigenvalues exactly equal to 1 — observed empirically (`0.829, 1.0, 1.0, 1.0, 1.0` in 3D).

8.4 Not MPI-aware

The solver assembles **dense local** matrices. When run with `mpirun -np ≥ 2`:

- each process sees only its **partition** of the mesh (`Model % NumberOfNodes`

and `Element % NodeIndexes` are **local**)

- the result **does not crash but is wrong**: `E(1) = 3.05` instead of `1.03`

In addition, Elmer 9.0 **cannot partition a mesh** on this platform (crash in `MeshPartition` → `i2s`), and the absence of `malha/partitioning.N/` triggers a `SIGSEGV` in `ModelDescription.F90:2895` (NULL deref).

Conclusion: do not use `mpirun` with this solver. For parallelism, export the matrices (§7) and solve with PETSc/SLEPc. Full analysis in `docs/PARALELISMO.md`.

8.5 VTU output limited to mode 1

The `psi` field receives **only the mode 1 eigenvector**. Attempts to publish all modes with `VariableAdd` (variables `psi_1`…`psi_N`) caused **heap corruption** (exit 134/139) and were reverted.
Alternative: solve the exported matrices with `resolve_exportado.py` to obtain all modes.

---

9. Error diagnostics

9.1 Symptom table

Symptom	Likely cause	Fix
`cannot open shared object file: SchrodingerSolver.so`	wrong `ELMER_SOLVER_HOME`, or `.so` not inst
`Unknown specifier ... property name:[potential]`	`Potential` placed in `Material`
`MATC ERROR: Index out of bounds`	`tx(1)` with only one variable declared
`Can't find independent variable`	variables separated by a **space**
`ElementMetric: Degenerate 3D element`	`Coordinate System` ≠ mesh dimension
No solver output at all	`Max Output Level < 4`
Unexpected `Sem potencial (V = 0)`	potential not read
`DSYGV retornou info = N`	diagonalisation failure
repeated `1.0` eigenvalues	Dirichlet penalisation
wrong result with `mpirun`	solver is not MPI-aware

9.2 Compilation errors

Error	Cause	Fix
`Symbol 'bf' already has basic type of DERIVED`	`BF`/`Bf` collision	r
`'listgetreal' has a type, not consistent with the CALL`	used as a subroutine	it
`Target expression in pointer assignment must deliver a pointer`	`=>` with a function	u
`'Info' at (1) has a type...`	local `INFO` collides	r
`Can't open included file 'mpif.h'`	wrong compiler in CMake	u

9.3 Reproducing the verification suite

```
# 5 1D cases with analytical solutions; expected: 5 passed, 0 failed
qsub -v CASO=... pbs/roda_caso_elmer.pbs
# or the full suite:
bash ~/schrodinger_P1_2/regressao.sh
```

---

10. Extending the solver

10.1 Making it MPI-aware (the ”correct” route)

This would require rewriting the assembly using Elmer’s native infrastructure:

```
CALL DefaultInitialize()
DO t = 1, Solver % NumberOfActiveElements      ! NEVER Model % NumberOfBulkElements
  Element => GetActiveElement(t)
  CALL DefaultUpdateEquations( STIFF, FORCE )  ! assembles K
  CALL DefaultUpdateMass( MASS )               ! assembles M
END DO
CALL DefaultFinishAssembly()
CALL DefaultDirichletBCs()
CALL DefaultSolve()
```

Critical points:

- the global matrix is created by the **driver**, not the solver

(`ElmerSolver.F90:803`); do not allocate `A(nDof,nDof)`

- `Solver % NumberOfActiveElements` ≠ `Model % NumberOfBulkElements`

(the current loop includes halo elements and double-counts)

- `Linear System Symmetric = Logical True` is **mandatory** so that

`CRS_ElimSymmDirichlet` (`SolverUtils.F90:8711`) runs and
`CRSMatrix.F90:1285` also zeroes the mass columns

- an ideal template already exists in Elmer: `modules/DFTSolver.F90` →

`WavefunctionSolver` (lines 218–505), which solves $(-\frac{1}{2}\nabla^2+V+Vxc)\psi=E\psi$
Estimated effort: 3–5 days. **Risk:** high — the native ARPACK eigensolver, in parallel, **converges but returns wrong eigenvalues** (tested on an SPD case with a known analytical solution: negative results).

10.2 Publishing all modes to the VTU

This requires fixing the heap corruption in `VariableAdd` (§8.5). The likely cause is the **ownership of the permutation array** — Elmer keeps the pointer,

and reusing the same `PermModos` for several variables may invalidate references. Alternatives: allocate one `Perm` per mode, or write one file per mode.

10.3 Radiation boundary condition

Barriers and resonant states would require implementing the Sommerfeld condition or *complex scaling*. Methodological reference: `00_planejamento/referencias/` — `qsolver` (CPC 329 (2026) 110393), which uses analytic mapping of the infinite domain plus polynomial collocation.

Appendix — quick reference

```
# COMPILE (headnode)
BLD=$HOME/elmer_build/build_mumps; INST=$HOME/elmer_build/install_mumps
cd ~/schrodinger_P1_2/build_mumps
mpif90 -c SchrodingerSolver.F90 -o SchrodingerSolver.o -fallow-argument-mismatch \
  -DCONTIG= -DHAVE_EXECUTECOMMANDLINE -DUSE_ISO_C_BINDINGS -O2 -fPIC -I$BLD/fmodules
mpif90 -shared SchrodingerSolver.o -o SchrodingerSolver.so -L$BLD/fem/src -lelmersolver
cp -f SchrodingerSolver.so $INST/share/elmersolver/lib/

# RUN (compute nodes!)
export ELMER_HOME=$HOME/elmer_build/install_mumps
export ELMER_SOLVER_HOME=$ELMER_HOME/share/elmersolver
export LD_LIBRARY_PATH=$ELMER_HOME/lib/elmersolver:/opt/ohpc/pub/libs/gnu12/openmpi4/mumps/5.2.1/lib:/
cd ~/P1_2_casos/casos/<case>
python3 gerar_malha.py
$ELMER_HOME/bin/ElmerSolver caso.sif > run.log 2>&1
grep "E(" run.log
paraview malha/run_t0001.vtu
```

Last updated: 2026-10-08